

Na Figura 9.3a, a execução da corrotina A é iniciada pela unidade mestra. Após alguma execução, A inicia B. Quando a corrotina B, na Figura 9.3a, faz com que o controle volte para a corrotina A, a semântica é que A continua a partir de onde terminou sua última execução. Em particular, suas variáveis locais têm os valores deixados nelas pela ativação anterior. A Figura 9.3b mostra uma sequência de execução alternativa das corrotinas A e B. Nesse caso, B é iniciada pela unidade mestra.

Em vez de ter os padrões mostrados na Figura 9.3, frequentemente uma corrotina possui um laço contendo uma retomada. A Figura 9.4 mostra a sequência de execução desse cenário. Nesse caso, A é iniciada pela unidade mestra. Dentro de seu laço principal, A retoma B, que, por sua vez, retoma A em seu laço principal.

Entre as linguagens contemporâneas, somente Lua suporta corrotinas integralmente.¹³

RESUMO

As abstrações de processo são representadas nas linguagens de programação por subprogramas. A definição de um subprograma descreve as ações representadas por ele. A chamada de um subprograma ativa essas ações. O cabeçalho de um subprograma identifica sua definição e fornece sua interface, denominada protocolo.

Parâmetros formais são os nomes utilizados pelos subprogramas para fazer referência aos parâmetros reais dados em chamadas de subprograma. Em Python e Ruby, parâmetros de vetor e formais de dispersão são usados para suportar números variáveis de parâmetros. Lua e JavaScript também suportam números variáveis de parâmetros. Os parâmetros reais podem ser associados aos parâmetros formais por posição ou por palavra-chave. Os parâmetros podem ter valores padrão.

Subprogramas podem ser funções, as quais modelam funções matemáticas e são usadas para definir novas operações, ou procedimentos, os quais definem novas sentenças.

Nos subprogramas, as variáveis locais podem ser dinâmicas da pilha, oferecendo suporte para recursão, ou estáticas, fornecendo eficiência e variáveis locais sensíveis ao histórico de execução.

JavaScript, Python, Ruby e Lua permitem aninhar definições de subprograma.

Existem três modelos de passagem de parâmetros fundamentais – modo de entrada, modo de saída e modo de entrada e saída – e várias estratégias para implementá-los. São elas: passagem por valor, passagem por resultado, passagem por valor-resultado, passagem por referência e passagem por nome. Na maioria das linguagens, os parâmetros são passados na pilha de tempo de execução.

A criação de apelidos pode ocorrer quando são usados parâmetros passados por referência, tanto entre dois ou mais parâmetros como entre um parâmetro e uma variável não local acessível.

Parâmetros que são matrizes multidimensionais apresentam alguns problemas para o projetista de linguagens, pois o subprograma chamado precisa saber como computar a função de mapeamento de armazenamento para eles. Isso exige mais do que apenas o nome da matriz.

¹³ Contudo, os geradores de Python são uma forma de corrotinas.

Os parâmetros que são nomes de subprograma fornecem um serviço necessário, mas podem ser difíceis de entender. A opacidade reside no ambiente de referenciamento que está disponível quando um subprograma passado como parâmetro é executado.

C e C++ suportam ponteiros para funções. C# tem representantes, os quais são objetos que podem armazenar referências para métodos. Os representantes podem oferecer suporte para chamadas multicast, armazenando mais de uma referência de método.

Ada, C++, C#, Ruby e Python permitem sobrecarga tanto de subprogramas como de operadores. Os subprogramas podem ser sobrecarregados contanto que possa ser feita a desambiguação das diversas versões pelos tipos de seus parâmetros ou valores retornados. Definições de função podem ser usadas para dar significados adicionais a operadores.

Em C++, Java 5.0 e C# 2005, os subprogramas podem ser genéricos, e o polimorfismo paramétrico pode ser usado para que os tipos desejados para seus objetos de dados possam ser passados para o compilador, o qual pode então construir unidades para os tipos solicitados.

O projetista de um recurso de função em uma linguagem deve decidir quais restrições serão impostas para os valores retornados e também o número de valores de retorno.

Um fechamento é um subprograma e seu ambiente de referenciamento. Fechamentos são úteis em linguagens que permitem subprogramas aninhados, têm escopo estático e possibilitam que subprogramas sejam retornados de funções e atribuídos a variáveis.

Uma corrotina é um subprograma especial que possui várias entradas. Pode ser usada para fornecer execução intercalada de subprogramas.

QUESTÕES DE REVISÃO

1. Quais são as três características gerais dos subprogramas?
2. O que significa para um subprograma estar ativo?
3. O que consta no cabeçalho de um subprograma?
4. Qual característica dos subprogramas Python os diferencia dos de outras linguagens?
5. Quais linguagens permitem um número variável de parâmetros?
6. O que é um parâmetro de vetor formal em Ruby?
7. O que é um perfil de parâmetros? O que é um protocolo de subprograma?
8. O que são parâmetros formais? O que são parâmetros reais?
9. Quais são as vantagens e desvantagens dos parâmetros de palavra-chave?
10. Quais são as diferenças entre uma função e um procedimento?
11. Quais são as questões de projeto para subprogramas?
12. Quais são as vantagens e desvantagens das variáveis locais dinâmicas?

13. Quais são as vantagens e desvantagens das variáveis locais estáticas?
14. Quais linguagens permitem aninhar definições de subprograma?
15. Quais são os três modelos semânticos de passagem de parâmetros?
16. Quais são os modos, os modelos conceituais de transferência, as vantagens e as desvantagens dos métodos de passagem de parâmetros por valor, por resultado, por valor-resultado e por referência?
17. Descreva as maneiras pelas quais podem ocorrer apelidos com parâmetros passados por referência.
18. Qual é a diferença entre a maneira como C original e C89 lidam com um parâmetro real cujo tipo não é idêntico ao do parâmetro formal correspondente?
19. Quais são as duas considerações de projeto fundamentais para métodos de passagem de parâmetros?
20. Descreva o problema da passagem de matrizes multidimensionais como parâmetros.
21. Qual é o nome do método de passagem de parâmetros usado em Ruby?
22. Quais são as duas questões que surgem quando nomes de subprograma são parâmetros?
23. Defina *vinculação rasa* e *profunda* para ambientes de referência de subprogramas passados como parâmetros.
24. O que é um subprograma sobrecarregado?
25. O que é polimorfismo paramétrico?
26. O que faz uma função template C++ ser instanciada?
27. De que maneiras fundamentais os parâmetros genéricos para um método genérico Java 5.0 diferem dos parâmetros genéricos dos métodos C++?
28. Se um método Java 5.0 retorna um tipo genérico, que tipo de objeto é retornado na realidade?
29. Se um método genérico Java 5.0 é chamado com três parâmetros genéricos diferentes, quantas versões do método serão geradas pelo compilador?
30. Quais são as questões de projeto para funções?
31. Quais são as duas linguagens que permitem retornar vários valores de uma função?
32. O que exatamente é um representante?
33. Qual é a principal desvantagem das funções genéricas em F#?
34. O que é fechamento?
35. Quais são as características da linguagem que tornam os fechamentos úteis?
36. Quais linguagens permitem ao usuário carregar operadores?
37. De que maneiras as corrotinas são diferentes dos subprogramas convencionais?

PROBLEMAS

1. Quais são os argumentos contra e a favor do fato de um programa de usuário criar definições adicionais para operadores existentes, como acontece em Python e C++? Você acha tal sobrecarga de operador definida pelo usuário boa ou ruim? Justifique sua resposta.
2. Na maioria das implementações de Fortran IV, todos os parâmetros eram passados por referência, usando apenas transmissão de caminho de acesso. Cite as vantagens e desvantagens dessa escolha de projeto.
3. Argumente a favor da decisão dos projetistas de Ada 83 de permitir que o implementador escolha entre implementar parâmetros de modo de entrada e saída por cópia ou por referência.
4. Suponha que você queira escrever um método que imprima um cabeçalho em uma nova página de saída, junto com um número de página que é 1 na primeira ativação e aumenta em 1 a cada ativação subsequente. Isso pode ser feito sem parâmetros e sem referência para variáveis não locais em Java? Isso pode ser feito em C#?
5. Considere o programa a seguir, escrito em sintaxe C:

```
void swap(int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

void main() {
    int value = 2, list[5] = {1, 3, 5, 7, 9};
    swap(value, list[0]);
    swap(list[0], list[1]);
    swap(value, list[value]);
}
```

Para cada um dos seguintes métodos de passagem de parâmetros, quais são todos os valores das variáveis `value` e `list` após cada uma das três chamadas a `swap`?

- a. Passados por valor
 - b. Passados por referência
 - c. Passados por valor-resultado
6. Apresente um argumento contra o fornecimento de variáveis locais estáticas e dinâmicas em subprogramas.
 7. Considere o programa a seguir, escrito em sintaxe C:

```
void fun (int first, int second) {
    first += first;
    second += second;
```

```

}
void main() {
    int list[2] = {1, 3};
    fun(list[0], list[1]);
}

```

Para cada um dos seguintes métodos de passagem de parâmetro, quais são os valores do vetor `list` após a execução?

- a. Passado por valor
 - b. Passado por referência
 - c. Passado por valor-resultado
8. Argumente contra o projeto de C de fornecer somente subprogramas.
 9. Com base em um livro-texto sobre Fortran, aprenda a sintaxe e a semântica das funções de declaração. Justifique a existência delas em Fortran.
 10. Estude os métodos de sobrecarga de operadores definida pelo usuário em C++ e Ada e escreva um relatório comparando os dois com base em nossos critérios de avaliação de linguagens.
 11. C# suporta parâmetros de modo de saída, mas nem Java nem C++ o fazem. Explique a diferença.
 12. Pesquise o dispositivo de Jensen, que foi um uso de parâmetros passados por nome, e redija uma breve descrição do que ele é e como pode ser usado.
 13. Estude os mecanismos iteradores de Ruby e CLU e liste suas semelhanças e diferenças.
 14. Reflita sobre subprogramas aninhados nas linguagens de programação – por que eles não são permitidos em muitas linguagens contemporâneas?
 15. Liste pelo menos dois argumentos contra o uso de parâmetros passados por nome.
 16. Escreva uma comparação detalhada dos subprogramas genéricos de Java 5.0 e C# 2005.

EXERCÍCIOS DE PROGRAMAÇÃO

1. Escreva um programa, em uma linguagem que você conheça, para determinar a razão do tempo exigido para passar um vetor grande por referência e o tempo exigido para passar o mesmo vetor por valor. Torne o vetor tão grande quanto possível na máquina e na implementação usadas. Passe o vetor quantas vezes for necessário para obter cronometragens razoavelmente precisas das operações de passagem.
2. Escreva um programa em C# ou Ada que determine quando o endereço de um parâmetro de modo de saída é computado (no momento da chamada ou no momento em que a execução do subprograma termina).